Build your FloodWatch device

React + Vite (TypeScript) | ASP.NET Core .NET 10 (C#) | ESP32 firmware (C++)

Build and test in this order: **USB power -> sensor -> GPS -> buzzer -> firmware -> dashboard -> solar.** Start with a tabletop or water-container test. The example wiring is for a classic ESP32-WROOM DevKit, not an ESP32-C3/S3 or an unverified module variant.

What to prepare

Main parts	Supporting parts and tools
ESP32 DevKit; JSN-SR04T sensor and PCB; NEO-6M breakout and antenna; small 5 V active buzzer (20 mA or less).	Breadboard, M-M/M-F jumpers, USB data cable, multimeter, ruler, computer and a small test container.
10-20 W nominal 12 V panel; 20,000 mAh power bank; weatherproof enclosure and rigid bracket.	Documented compatible solar-to-5 V USB stage; fused/switchable USB distribution; glands and strain relief.
SN74AHCT125, 14-pin package or labeled adapter; NPN transistor with identified B/C/E pins.	10 kOhm resistors x3, 18 kOhm x1, 1 kOhm x1 (1/4 W); 100 nF capacitor x1; 1N4007 diode for electromagnetic buzzer.

Understand the connections

Red: regulated +5 V. **Black:** common ground. **Orange:** trigger. **Green:** echo. **Blue:** GPS TX. **Purple:** buzzer control. Label both ends of each wire. Use the GPIO labels printed on the ESP32, not the physical pin position in a random board picture.

Step 1. Inspect first, then wire with power OFF

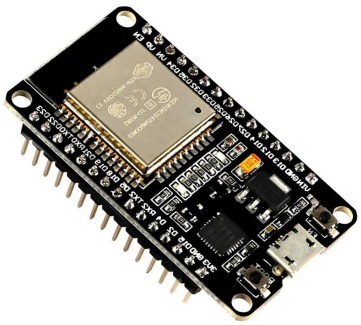
1. Record the sensor revision and confirm TRIG/ECHO pulse mode. Confirm the GPS breakout input voltage and UART levels.
2. Identify the transistor base, collector and emitter from its actual datasheet.
3. Identify the breadboard power rails. Some rails are split in the middle; bridge only the sections that must share the same voltage.
4. Build the GND rail, then the protected +5 V rail. Do not connect power yet.
5. Follow the next four wiring pages. Keep the solar panel disconnected until the complete USB-powered assembly passes testing.

One supply at a time: During programming, power the circuit from a single PC USB connection through an appropriate distribution/breakout arrangement. Disconnect the power bank. Never connect a PC USB supply and an external 5 V supply together through the ESP32. Before connecting the board, use a meter to check polarity and short circuits.

Build status: The web application exists locally; the firmware below is a starter. No physical device has been assembled or validated here. Final wiring and solar compatibility depend on the purchased component versions.

Meet the main components

Actual supplier photos for identification, confirm the exact model before buying

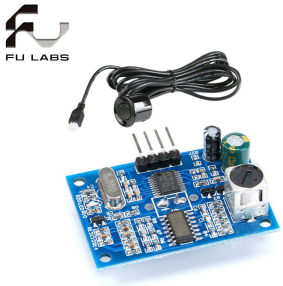


1. ESP32-WROOM DevKit

Estimate: PHP 250-450

Main controller. Photo shows a 30-pin Micro-USB development board.

Photo: Fu-labs | product reference

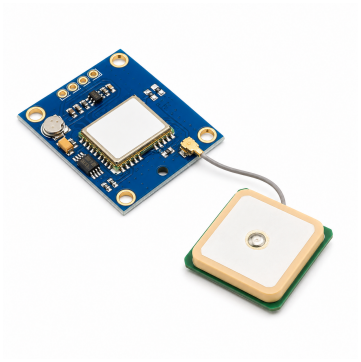


2. JSN-SR04T sensor set

Estimate: PHP 220-600

Buy the complete board, cable and waterproof probe. Confirm pulse mode.

Photo: Fu-labs | product reference



3. NEO-6M GPS + antenna

Estimate: PHP 400-750

Station location receiver. Check breakout supply voltage before wiring.

Photo: Circuitrocks | product reference



4. Small 5 V active buzzer

Estimate: PHP 20-100

Two-pin buzzer, not a passive sounder. Photo shows a two-piece pack.

Photo: PowerMav | product reference

Use these photos to recognize parts, not as pin-position diagrams. Boards may look different across suppliers. Follow the labeled wiring pages and actual component datasheets. The displayed price ranges are planning estimates for one required item or pack, not a quotation for the pictured brand.

Power, protect and connect

Representative photos; compatibility and enclosure fit still need to be checked



5. 20,000 mAh power bank

Estimate: PHP 1,000-2,500

Example appearance only. Verify pass-through, restart and low-load output.

Photo: UGREEN | product reference

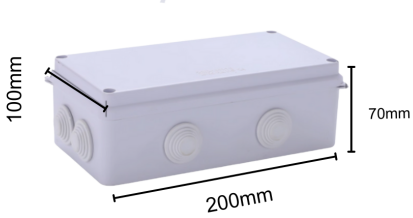


6. Monocrystalline solar panel

Estimate: PHP 800-3,500

10 W kit shown for identification. Select a compatible 10-20 W panel.

Photo: DFRobot KIT0210 | product reference



7. Weatherproof enclosure

Estimate: PHP 160-600

Photo: 200 x 100 x 70 mm example. Measure parts and cable clearance first.

Photo: Circuitrocks | product reference



8. M-M and M-F jumper wires

Estimate: PHP 75-180

Photo shows M-F jumpers. Buy both end types for the breadboard build.

Photo: Circuitrocks | product reference

Power compatibility: The pictured UGREEN bank is not a verified pass-through recommendation. The DFRobot kit includes a different controller/battery arrangement and a 5 V/1 A output; it is shown for panel identification, not as a direct replacement for our 5 V/2 A-or-higher bank plan. Do not copy the kit wiring into the separate power-bank design.

Also required but not pictured: solar-compatible USB regulation, fuse/switch distribution, logic buffer, transistor, resistors, capacitor, optional protection diode, breadboard/perfboard, USB cable, mounting hardware and cable glands. Include all of them in the budget below.

BUDGET | ONE COMPLETE STATION

Possible cost of the build

PHP | Budget allowances checked against sample listings on 22 October 2026

Item	Qty	Low	High
ESP32-WROOM development board	1	250	450
JSN-SR04T board + waterproof probe	1 set	220	600
NEO-6M GPS + antenna	1 set	400	750
Small 5 V active buzzer	1 / pack	20	100
20,000 mAh qualified power bank	1	1,000	2,500
10-20 W monocrystalline solar panel	1	800	3,500
Compatible solar-to-5 V USB regulator	1	600	1,500
Weatherproof enclosure	1	160	600
M-M + M-F jumper wires	2 packs	75	180
Buffer, transistor, resistors, capacitor, diode	1 lot	150	350
Perfboard, fuse/switch, terminals, distribution	1 lot	300	650
Cable glands, insulation and strain relief	1 lot	200	450
Mounting bracket / support materials	1 lot	400	1,000
Breadboard	1	100	200
USB data cable	1	100	200
Subtotal		4,775	13,030
15% contingency (rounded up)		717	1,955
Estimated complete station		5,492	14,985

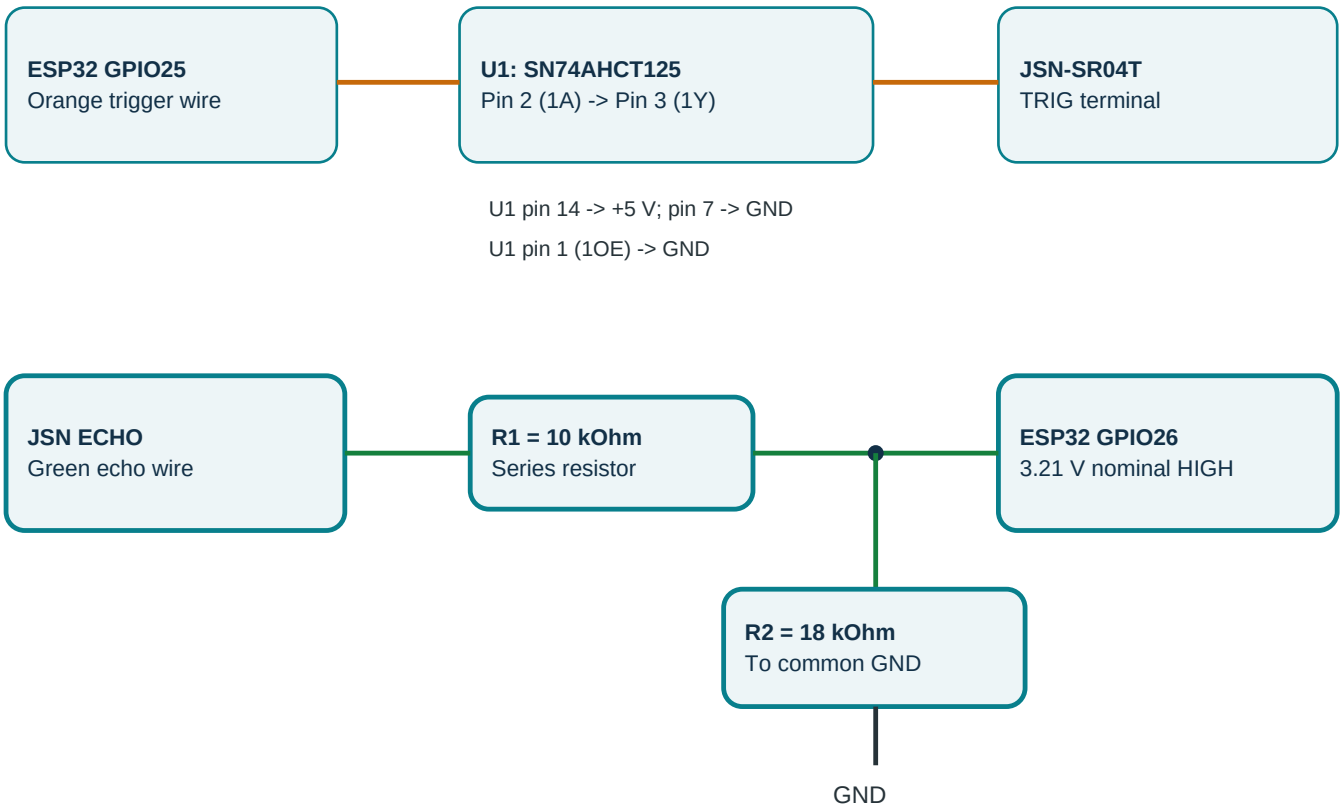
Suggested working budget: PHP 10,000-12,000, then confirm every supplier quote and the power-chain compatibility. Higher-grade solar or charging hardware can exceed this allocation. Existing computer, router, internet, multimeter and soldering tools are assumed available. Allow an extra PHP 800-2,000 for basic tools if needed. Hosting, mobile data, labor and printing are excluded.

Price examples, not guaranteed availability: Fu-labs ESP32-30P-MICRO-CP2102 PHP 284 and JSN-SR04T listing from PHP 220; Circuitrocks GPS PHP 692; PowerMav 2-piece buzzer pack PHP 20; Circuitrocks 20 cm M-M/M-F wire packs PHP 40/PHP 35. Variant choice changes price. Panel, power bank and compatible regulator figures remain planning allowances.

Sources: Click each gallery photo credit for its supplier page. Photos belong to the named suppliers/manufacturers. The budget is for a generic compatible build, not the exact pictured brands or a certified field installation. The solar image is from the complete KIT0210 package; its kit price is not the panel-only allowance.

JSN-SR04T detailed wiring

Figure B1. Two separate signal paths: boosted TRIG and reduced ECHO

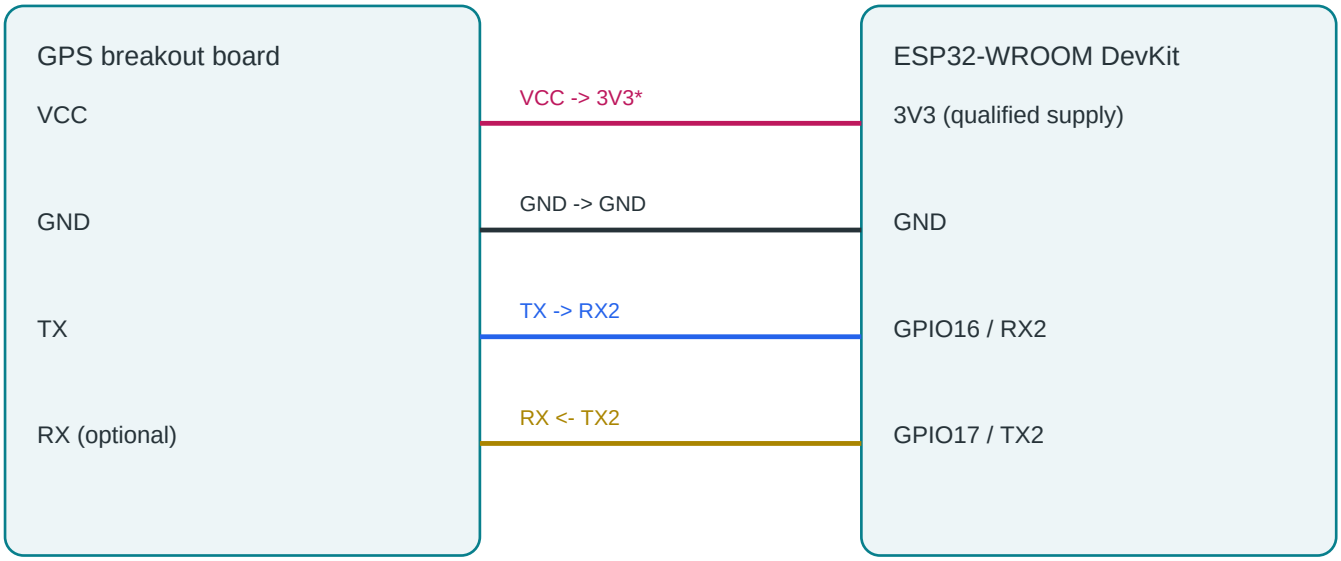


Wire / component	Exact terminal connection
Sensor power	JSN VCC -> fused +5 V rail; JSN GND -> GND rail. Sensor head plugs into its own PCB connector, not ESP32 pins.
U1 supply and bypass	Use a 14-pin SN74AHCT125 package with verified pin numbers [7]. Pin 14 -> +5 V, pin 7 -> GND; place 100 nF between pins 14 and 7.
U1 used channel	GPIO25 -> pin 2. Pin 3 -> sensor TRIG. Pin 1 -> GND. Add 10 kOhm from pin 2 to GND to hold TRIG low during reset.
U1 unused channels	Pins 4, 10, 13 (unused enables) -> +5 V. Pins 5, 9, 12 (unused inputs) -> GND. Leave output pins 6, 8, 11 unconnected.
ECHO junction	Sensor ECHO -> R1 -> junction -> GPIO26. Junction -> R2 -> GND. Use 1% resistors. Never connect ECHO directly to GPIO26.

Divider: Vout = Vin x 18 / (10 + 18). A 5.0 V input becomes about 3.21 V. Confirm the actual sensor supply and echo amplitude before connection. Verify the exact sensor revision is configured for TRIG/ECHO pulse mode. Use an oscilloscope or logic-level test where available; a multimeter may not capture short echo pulses.

NEO-6M UART connections

Figure C1. TX must connect to RX, not TX to TX



*** Supply gate:** This drawing assumes a breakout explicitly documented to operate from a 3.3 V input. Some NEO-6M boards include an onboard regulator and may require a different VCC input. If its documentation requires 5 V VCC, connect only VCC to the regulated +5 V rail and verify its UART output remains 3.3 V compatible. Do not guess from the board color. The bare NEO-6M module supply limit differs from a breakout input [2].

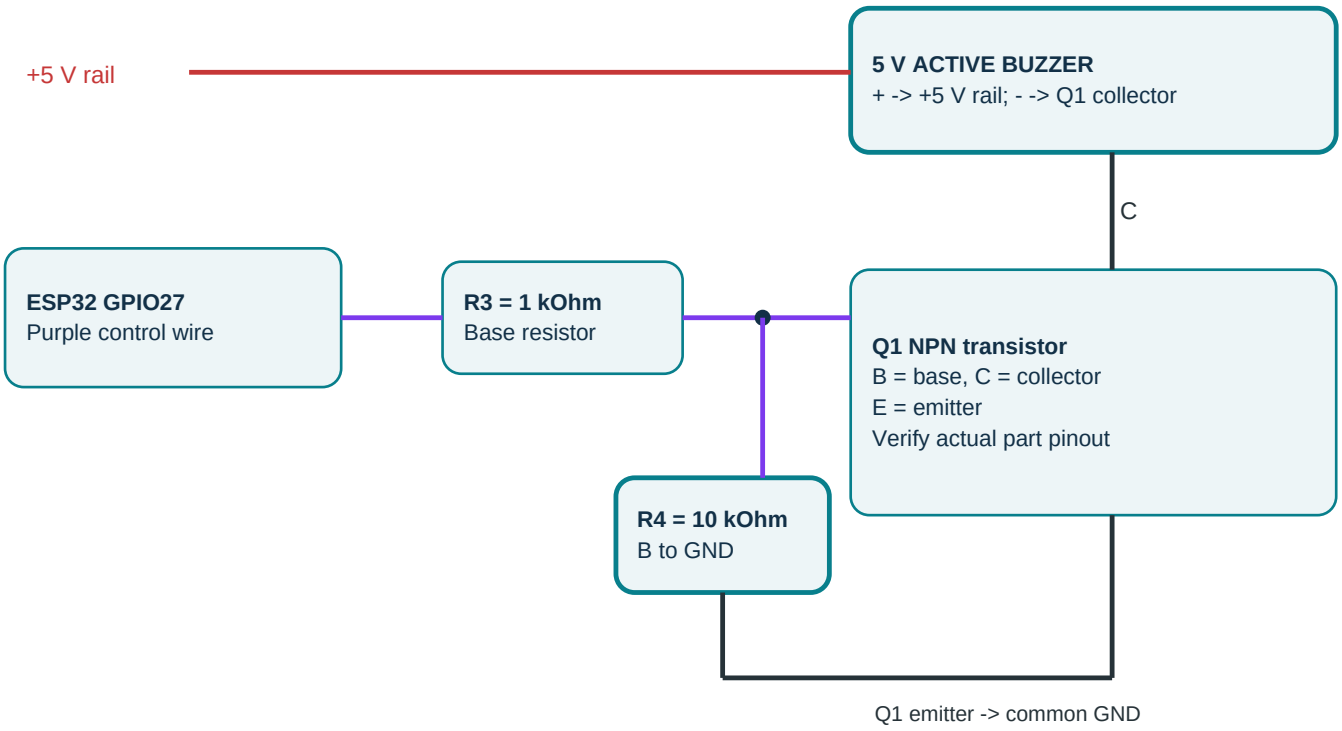
Wire ID	From	To
G1, pink	ESP32 3V3, only if board supports it	GPS VCC
G2, black	ESP32/common GND	GPS GND
G3, blue	GPS TX	ESP32 GPIO16 (RX2)
G4, yellow, optional	ESP32 GPIO17 (TX2)	GPS RX
Antenna	Supplied antenna connector	Matching GPS antenna socket

Student steps: (1) Disconnect all power. (2) Verify VCC requirements and label the supply wire. (3) Connect ground. (4) Connect GPS TX to GPIO16. (5) Leave GPS RX unconnected unless firmware needs to configure the receiver. (6) Attach its antenna carefully, then power on and test outdoors. Configure UART baud to match the actual module, commonly 9600 baud, 8-N-1.

A valid UART sentence is not automatically a valid location fix. Show no-fix/old-fix status in firmware and the dashboard. Keep the antenna away from the metal bracket and do not place it below the solar panel.

Active buzzer transistor driver

Figure D1. Example for a 5 V two-wire active buzzer drawing no more than 20 mA



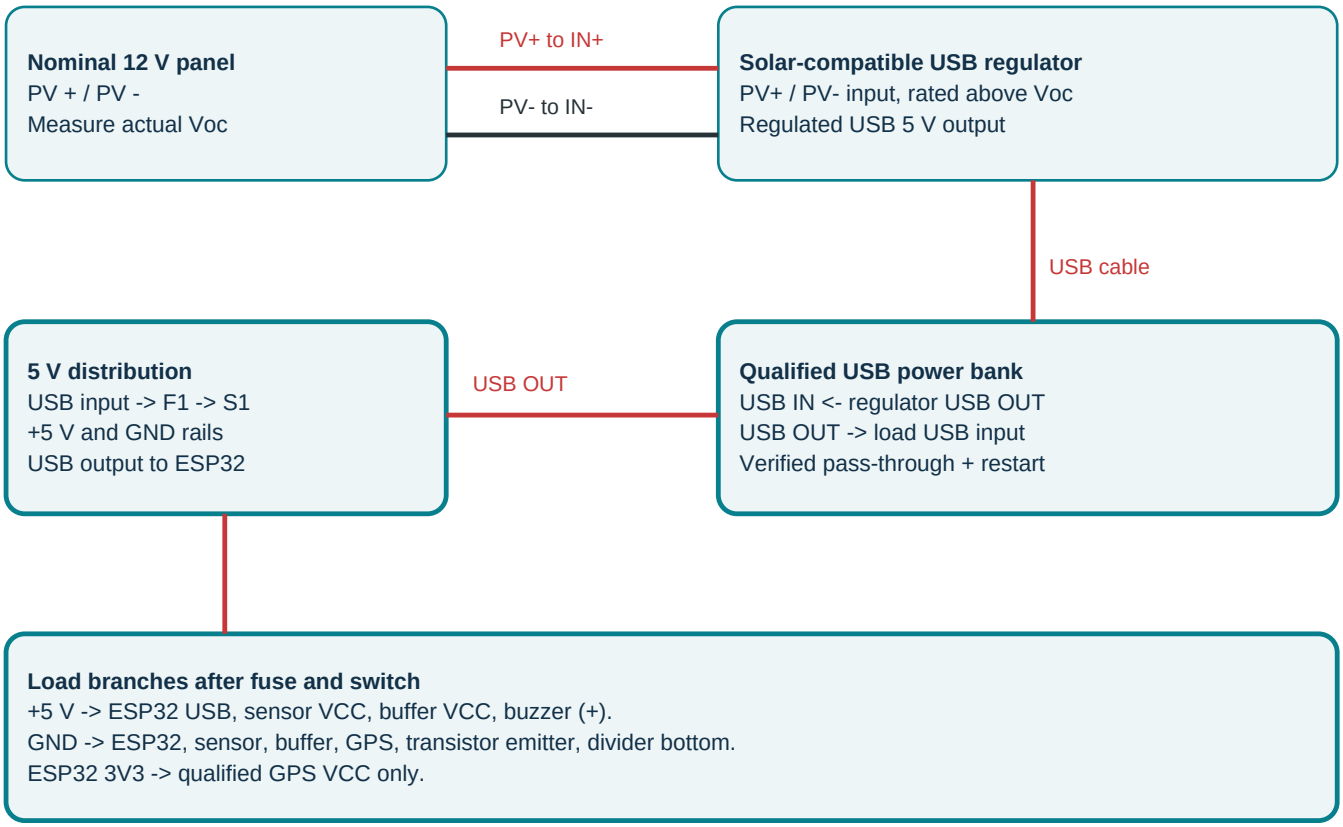
Wire / part	Connection
B1, purple	GPIO27 -> 1 kOhm resistor R3 -> transistor BASE.
B2, black	Transistor EMITTER -> common GND.
B3, black	Buzzer negative (-) -> transistor COLLECTOR.
B4, red	Buzzer positive (+) -> regulated +5 V. Use only a buzzer rated for this voltage.
R4 pull-down	10 kOhm between BASE (after R3) and GND. Keeps buzzer off during ESP32 reset.
D1 protection	For an electromagnetic buzzer: diode across buzzer, striped cathode -> buzzer (+), anode -> buzzer (-)/collector. Example: 1N4007.

GPIO27 HIGH turns Q1 on and sounds the active buzzer. LOW turns it off. A two-wire active buzzer makes its own tone; an intermittent warning is made by switching it on/off in firmware. A three-pin buzzer module has a different circuit and must follow its own documentation.

For a small load, use a genuine NPN such as P2N2222A with its manufacturer pinout [8]. Do not assume that all parts marked 2N2222 have the same left-to-right legs. Identify B/C/E using the actual datasheet and a meter before soldering. This 1 kOhm drive example is limited to a measured buzzer current of 20 mA or less; redesign the driver for a larger alarm.

Power wiring, terminal by terminal

Figure E1. Required regulated power chain, selected hardware must be qualified first



Wire ID	Exact connection / check
P1 / P2	Panel + -> regulator PV+/IN+; panel - -> PV-/IN-. Use correct mating terminals. Do not reverse polarity.
P3 / P4	Regulator USB OUT -> power-bank USB IN using an intact USB cable. Bank USB OUT -> distribution USB input. Do not connect two output ports together.
F1 / S1	Distribution +5 V -> fuse F1 -> switch S1 -> +5 V load rail. Initial fuse candidate: 1 A, only after measuring load/inrush and confirming cable ratings. GND stays continuous.
ESP32 power	Use a USB breakout/distribution board that passes switched 5 V and GND to the ESP32 USB connector. Do not also feed VIN/5V or plug in a powered PC simultaneously.
Limits	The PV input terminals are generic functional names, not a promised module pinout. Match the purchased regulator labels. Do not use a bare buck board as an assumed solar charger.

The exact solar regulator and power-bank models have not been supplied. Therefore this is the required connection topology, not a verified product-specific charger schematic. Before buying, require documentation covering panel Voc, USB output regulation and bank pass-through behavior. If that pair cannot sustain stable operation, select a matched solar UPS/battery system and revise this page. Do not open or rewire cells inside a consumer power bank.

Upload firmware and connect

Frontend: React + Vite / TypeScript | Backend: ASP.NET Core / C#

Step 6. Prepare Arduino IDE

Install Arduino IDE from its official website. In Preferences, add the ESP32 Boards Manager URL from the Espressif installation guide [1]. Install **esp32 by Espressif Systems**. Select the actual classic ESP32 board and its COM port. In Library Manager, install **TinyGPSPlus by Mikal Hart**. Use a USB data cable, not a charge-only cable.

Step 7. Set your firmware values

Open **firmware/FloodWatch/FloodWatch.ino**. Copy **config.example.h** to **config.h** in the same folder. Enter your 2.4 GHz Wi-Fi name/password, computer API address and shared device key. Do not share config.h. Set H and the thresholds to match the dashboard. Default pins: TRIG 25, ECHO 26, GPS RX 16, optional GPS TX 17, buzzer control 27.

Setting	Example / meaning
MOUNTING_HEIGHT_CM	150.0: sensor face to fixed reference datum
ADVISORY / WARNING / CRITICAL	30 / 50 / 70 cm: laboratory examples only
API_URL	http://192.168.1.10:5086/api/readings Replace the sample IP with the actual computer private IPv4.
DEVICE_KEY	Same private value as backend DEVICE_API_KEY
SENSOR_MIN_CM	30 cm initially; increase if required by actual sensor

Step 8. Start the API for a supervised LAN test

On the computer, open PowerShell in the project folder. Set the following variables, then run the backend. This deliberately enables LAN access; do it only on a trusted private network. If Windows requests access, limit it to that private network. Do not port-forward the API.

```
$env:FLOODWATCH_URL = 'http://0.0.0.0:5086'
$env:DEVICE_API_KEY = 'your-long-random-key'
dotnet run --project backend/FloodWatch.Api.csproj
```

Use **ipconfig** to find the computer's private IPv4 address. The ESP32 and computer must be on the same Wi-Fi with client isolation disabled by the network owner. Firmware must use that IP, not localhost. Internet access is needed for the starter's NTP clock synchronization before telemetry begins.

In a second terminal, run **cd frontend**, **pnpm install** (first setup), then **pnpm dev**. Open **http://localhost:5186** and turn **Demo mode OFF**. Upload the sketch and open Serial Monitor at **115200 baud**. A valid API POST should report HTTP 201; a duplicate packet can return 200.

Test it before the final enclosure

Use a safe test container, not active floodwater

Step 9. Calibrate and confirm the entire path

Measure H from the sensor face to a fixed datum. Point the sensor straight down. Keep the water at least the verified minimum sensor clearance away from the face. Record a ruler reference at five known levels. Example: H = 150 cm and measured gap = 90 cm give level = 60 cm. Change both firmware H and dashboard H together, then re-upload firmware.

Test	Expected observation
Sensor and GPS	Serial Monitor shows changing distance/level. GPS coordinates appear only after a recent valid fix; test with a clear sky view.
Local alarm	Hold above each configured threshold for several sample cycles. Warning/critical sound different patterns. Keep alarm tests short.
Dashboard integration	With Demo mode OFF, actual device readings appear in history and the chart. Verify level agrees with the serial output.
Invalid sensor / Wi-Fi loss	Remove the target or disconnect Wi-Fi. Invalid samples are not sent. Local warning logic continues; dashboard becomes stale after 30 seconds.
Power bank and solar	After USB success, disconnect the PC and use bank power. Connect only the qualified solar regulator to the bank input. Watch for resets during shade and input changes.

Step 10. Finish the installation

Transfer the successful bench circuit to secured soldered wiring or a PCB. Fit glands, fuse, switch, strain relief and drip loops. Protect the sensor control PCB, GPS board and battery inside the enclosure. Keep the sensor head facing downward, the antenna with sky view and the battery shaded. Recheck H after mounting. Run a supervised backup-power trial and log actual runtime before claiming solar autonomy.

Troubleshooting and starter limitations

HTTP -1 / connection failure: check computer IP, port 5086, private-network firewall permission and Wi-Fi isolation.
401: shared key mismatch. **400:** invalid distance, timestamp or coordinates. **503:** server key not configured.
No uploads: NTP may not be synchronized. Local sensing still runs.
Always invalid: check echo divider, common GND, sensor mode and blind zone.
Resets: measure supply under load; inspect bank low-load cutoff.

Starter firmware sends every 10 seconds when valid and time-synchronized. It has no offline replay or remote configuration. Network work runs separately from sensing; echo acquisition has a bounded timeout. Fault cycles retain the last hazard state. Firmware compilation and real-device behavior still require Arduino IDE and hardware validation.

References: [1] Espressif installation guide; [2] u-blox NEO-6 datasheet; [7] TI buffer datasheet; [8] onsemi transistor datasheet. Reference numbers match the wiring sheets. Always use the exact purchased component documentation.

SOFTWARE REFERENCE | ACTUAL PROJECT STACK

Languages and development tools

The languages and frameworks used in the current local FloodWatch project

Part of the system	Language	Framework / tool
Web frontend	TypeScript (TSX), HTML and CSS	React 19 + Vite 7
Backend / API	C#	ASP.NET Core on .NET 10
ESP32 firmware	C++ (Arduino sketch)	Arduino framework for ESP32
Data exchange	JSON format	HTTP requests between device, API and frontend
Current data storage	JSON files, not a SQL database	Local backend/Data folder

Frontend: the interface students will use

The frontend is written in **TypeScript**, with **React** components for the dashboard, charts, reading history, alert activity and station settings. **Vite** runs the development server and creates the production build. **CSS** controls the layout, colors, text sizes and mobile responsiveness. React and Vite are tools/frameworks, not programming languages.

Backend: the API that receives and stores data

The backend is written in **C#** using **ASP.NET Core on .NET 10**. It receives ESP32 readings, checks the device key and input values, calculates water level, prevents duplicate records, and saves accepted data. The React frontend retrieves readings from this API every 10 seconds.

Device firmware: the program inside the ESP32

The companion starter is written in **C++** as an **Arduino .ino sketch**. It reads the ultrasonic sensor and GPS, controls the buzzer, connects to Wi-Fi and sends JSON telemetry to the backend. It must still be compiled in Arduino IDE and validated on the actual device.

How the three parts connect

ESP32 / C++ measures the water level and sends an HTTP request containing JSON data.
ASP.NET Core / C# validates the request and saves the reading.
React + Vite / TypeScript requests the saved data and displays it in the browser.

Suggested wording for the thesis

The web interface was developed using TypeScript and React, with Vite as the development and build tool. The backend was developed using C# and ASP.NET Core on .NET 10. The ESP32 firmware starter uses C++ with the Arduino framework. The system exchanges data through HTTP requests in JSON format. The current prototype stores readings in local JSON files.

Project folders: **frontend/** for the web interface, **backend/** for the API, and **firmware/FloodWatch/** for the device program. This describes the current implementation; a database server, SignalR, public hosting and physical-device validation have not been completed.